VELMORA CO. ENTERPRISE SUITE · TEAM 3 ENGINEERING

Velmora Production-FMS Architecture Reference

Complete Function-by-Function Source Code & Business Logic Manual for Developers

TARGET STACK	DATABASE	MODULE SCOPE	DOCUMENTATION STATUS
Python 3.11 / Flask / PostgreSQL	Supabase / psycopg2 Pool	Production & Manufacturing (FMS)	Production-Verified (001-005 Schema)

1. Executive Architecture Overview

The Velmora Production-FMS module automates the entire manufacturing workflow for bespoke furniture production. It enforces a strict, unidirectional 6-stage lifecycle that prevents material waste, maintains live inventory synchronization, tracks workshop craftsmanship, and accurately accounts for direct manufacturing costs.

```
[1. Product Master & BOM] → Defines furniture specifications and raw material recipes (with waste factor). ↓ [2. Production Planning] → Schedules target start/due dates and binds the batch to workshop craftsmen. ↓ [3. Material Readiness] → Compares required components with warehouse stock; prevents releases on shortage. ↓ [4. Material Issuance] → Deducts physical inventory from prod_material and auto-initializes workshop tracking at 0%. ↓ [5. Workshop Monitoring] → Workers progress batches through 0% → 25% → 50% → 75% → 90% with proof photos. ↓ [6. Quality Inspection] → Passed units credit to Finished Goods. Defective units trigger Rework (75% rollback, max 2). ↓ [7. Job Costing & Reports] → Aggregates actual material cost + direct carpentry labor. Overhead managed by Sales.
```

2. Application Core & Database Layer

app.py

APPLICATION FACTORY & ROOT ENTRY POINT

Initializes the Flask application, registers the production Blueprint under `/production`, and enforces request-level database cleanup.

Function / Handler	Signature	Description & Engineering Purpose
FUNC **root**	`root() → Response`	Catches incoming requests to the domain root (/) and immediately issues an HTTP 302 redirect to the production home route (`/production/`).
FUNC **closeRequestContextDb**	`closeRequestContextDb(error=None)`	Decorated with `@app.teardown_appcontext`. Guarantees that any database connection attached to Flask's `g.dbConnection` during request handling is safely returned to the pool, automatically rolling back uncommitted transactions if an unhandled exception occurred.

production/db.py

POSTGRESQL THREADED CONNECTION POOL & TRANSACTIONS

Manages persistent SSL connections to Supabase PostgreSQL using `psycopg2.pool.ThreadedConnectionPool` and provides transaction context managers.

Function	Signature	Description & Engineering Purpose
FUNC **getPool**	`getPool() → ThreadedConnectionPool`	Initializes a thread-safe singleton connection pool (min 1, max 10 connections) configured with `RealDictCursor` so query results can be indexed like Python dictionaries. Enforces `sslmode=require` for Supabase cloud security.
FUNC **checkoutConnection**	`checkoutConnection() → connection`	Pulls an active socket connection from the pool. If a connection was closed by the remote server during idle periods, safely purges it and checks out a fresh connection.
FUNC **releaseConnection**	`releaseConnection(conn, error=False)`	Returns a connection back to the pool. If `error=True`, issues a rollback to ensure no partial state leaks into future requests; otherwise commits and returns.
FUNC **openTransaction**	`openTransaction(externalConnection=None)`	Context manager (`with openTransaction() as conn:`). Enables connection reuse across multiple domain models inside a single web request using Flask's `g.dbConnection`. Handles automatic commit on block completion and rollback on exception.

production/cache.py

REQUEST & IN-MEMORY DATA CACHING

Provides light in-memory caching to eliminate redundant database roundtrips for semi-static reference data.

Function	Signature	Description & Engineering Purpose
FUNC **getCached / setCached**	`getCached(key), setCached(key, val)`	Retrieves or stores timestamped cached query objects in memory.
FUNC **invalidateKeys / clearAllCache**	`invalidateKeys(), clearAllCache()`	Invalidates specific cache namespaces (e.g., when new furniture is created) or flushes the cache entirely.

3. Security & Enterprise Authentication Layer

`production/security.py`

BRUTE-FORCE RATE LIMITING & 24-HOUR IP BAN

Protects the login route from credential stuffing without altering any Supabase SQL schemas. Persists state thread-safely in `instance/login_ratelimit.json`.

Function	Signature	Description & Engineering Purpose
FUNC `get_client_ip`	`get_client_ip() → str`	Extracts the client's actual remote IP address, inspecting `X-Forwarded-For` (first proxy IP) before falling back to `remote_addr`.
FUNC `check_ip_security`	`check_ip_security(ip) → (bool, str, int)`	Evaluates whether the client IP is blocked. Returns `(is_allowed, reason, seconds_remaining)`. Blocks users if under an active 60s cooldown or 24-hour ban.
FUNC `record_failed_login`	`record_failed_login(ip) → (bool, str, int)`	Increments the failed login count for an IP. Reaching 5 failures triggers a 60-second block; accumulating 3 consecutive blocks escalates to a 24-hour IP ban.
FUNC `record_successful_login`	`record_successful_login(ip)`	Resets failed login attempt counts upon valid authentication, leaving legitimate users unencumbered.

production/auth.py

SUPABASE AUTH, AUTO-PROVISIONING & ROLE ACCESS CONTROL

Manages employee role validation, Supabase `auth.users` bcrypt authentication, auto-provisioning, profile editing, and cooldown password resets.

Function	Signature	Description & Engineering Purpose
FUNC `is_authorized_production_employee`	`(dept_id, pos_id, is_active=True) → bool`	Validates that an employee belongs to Department 6 (Production) and Position 6 (Manager) or 2 (Production Staff) with an active status.
FUNC `authenticate_credentials`	`authenticate_credentials(email, pwd) → (dict, str)`	Authenticates email/password using `extensions.crypt(%s, a.encrypted_password)`. Features an **Auto-Provisioning Engine**: if an active employee exists in `public.employee` but has no login account yet, entering initial password `velmora123` automatically creates their `auth.users` record, hashes the password, and links permissions.
FUNC `_sync_employee_auth_links`	`(cur, emp_id, auth_uid, pos_id) → str`	Ensures consistent foreign key links across `public.employee_auth`, `public.users`, and `public.employee_module_access` (setting access level to `manager` or `staff`).
FUNC `sync_and_verify_session`	`sync_and_verify_session() → bool`	Runs on every page request to verify that the logged-in employee remains active in the database, automatically kicking deactivated accounts.
FUNC `update_employee_profile`	`(emp_id, first_name, last_name, phone) → (bool, str)`	Updates employee name and phone number in `public.employee`, validating name lengths and phone formatting.
FUNC `change_employee_password`	`(emp_id, cur_pwd, new_pwd) → (bool, str)`	Validates current password, enforces minimum 8 characters with letters/numbers, checks a 30-day cooldown period, and updates the bcrypt hash in `auth.users`.
FUNC `login_required / manager_required`	`decorator(view_func) → view_func`	Flask route decorators. `login_required` ensures valid session; `manager_required` restricts executive operations (approvals, costing, releases) to Managers (pos 6).

production/integration.py

CROSS-MODULE INTEGRATION HANDLERS

Connects the Production module with Team 2 Inventory and Team 1 Finance.

Function	Signature	Description & Engineering Purpose
FUNC get_material_stock / deduct_material_stock	(materialID, qty, conn)	Queries or deducts raw material quantities from prod_material, safeguarding against negative balances.
FUNC add_finished_goods_stock	(furnitureID, qty, conn)	Credits passed production units to the finished goods inventory upon QA inspection completion.
FUNC get_warehouse_stockouts	get_warehouse_stockouts(conn) → list	Scans warehouse stock for items below reorder thresholds and alerts production managers.

4. Domain Models & Business Logic

production/models/furniture.py

FINISHED GOODS & PRODUCT MASTER CATALOG

Class & Methods	Signature	Description & Engineering Purpose
CLASS Furniture	Furniture(data_dict)	Data entity wrapping catalog products (furniture_id, furniture_name, furniture_type, selling_price).
METHOD getAll	getAll(searchQuery, category, conn)	Returns all catalog items with their computed BOM cost and category filters.
METHOD getByID / existsByName	getByID(id, conn), existsByName(name, excludeID, conn)	Retrieves product by ID and prevents duplicate naming across the catalog.
METHOD addFurniture / updateFurniture	(name, type, price, conn)	Creates or modifies furniture catalog entries with transaction safety.
METHOD canDelete / deleteFurniture	canDelete(id, conn) → (bool, str)	Enforces referential integrity: blocks deletion if furniture has active BOM items, orders, or output history.

production/models/billofmaterials.py & bomitem.py

ENGINEERING BILL OF MATERIALS & COMPONENT COSTING

Class & Methods	Signature	Description & Engineering Purpose
CLASS BillOfMaterials	BillOfMaterials(data_dict)	BOM header entity linking a furniture piece to its raw material component recipe.
METHOD getByFurnitureID	(furnitureID, autoCreate=True, conn)	Fetches the active BOM header for a furniture piece, auto-creating a Version 1 record if none exists.
METHOD getItems / addMaterial / removeMaterial	(materialID, qtyRequired, conn)	Manages BOM line items (adding raw materials with required quantities, removing components).
METHOD calculateRequirements	(plannedQuantity, conn) → list	Multiplies BOM required quantities by planned batch units to forecast material requirements.
CLASS BOMItem	calculateQuantity(plannedQuantity)	Line-item entity representing a required material, unit cost, and standard scrap allowance.

production/models/material.py & materialissuance.py RAW MATERIALS & FLOOR STOCK ISSUANCE

Class & Methods	Signature	Description & Engineering Purpose
CLASS Material	getAll(conn), getByID(id, conn)	Entity wrapping prod_material, returning stock levels, unit costs, and categories (Lumber, Hardware, Finishing).
CLASS MaterialIssuance	recordIssuance(orderID, matID, qty, user, conn)	Creates an official warehouse dispatch record in prod_materialissuance when materials leave inventory for the floor.
METHOD getAll / getByOrderID	getByOrderID(orderID, conn) → list	Returns all historical material dispatch logs for audit reports and actual cost calculations.

production/models/productionorder.py PRODUCTION ORDERS, SCHEDULING & LIFECYCLE STATE MACHINE

Class & Methods	Signature	Description & Engineering Purpose
CLASS ProductionOrder	ProductionOrder(data_dict)	Master work order entity (status: Draft, Released, In Progress, Completed, Cancelled).
METHOD scheduleHealth	scheduleHealth → str	Calculates real-time schedule status: Overdue (past due date), Due Soon (due within 3 days), or On Track.
METHOD createOrder	(furnitureID, qty, start, due, purpose, conn)	Inserts a new Draft production batch work order into prod_productionorder.
METHOD updateStatus	(orderID, newStatus, conn)	Advances order through lifecycle transitions (e.g. Draft → Released → In Progress).
METHOD updateSchedule	(orderID, start, due, assignedTo, updateAssignment, conn)	Updates production calendar timeline dates and assigns a craftsman employee from the staff roster.
METHOD canDelete / deleteOrder	canDelete(orderID, conn) → (bool, str)	Restricts deletion: only Draft orders with zero material dispatches can be safely deleted.

production/models/monitoring.py & workflow.py

WORKSHOP FLOOR ASSEMBLY STAGES & STAFF WORKFLOW

Class & Methods	Signature	Description & Engineering Purpose
CLASS **ProductionMonitoring**	`updateProgress(monitoringID, progressVal, conn)`	Advances workshop assembly stages: `0% (Not Started) → 25% (Cutting) → 50% (Joinery) → 75% (Finishing) → 90% (Ready for QA)`.
METHOD **updateProgressAndRework**	`(monitoringID, progressVal, reworkCount, conn)`	Rolls back progress to 75% when QA fails with Rework status, incrementing the rework counter (max 2).
FUNC **isValidActiveProdStaff**	`(employeeID, conn) → bool`	Verifies that a selected craftsman is an active employee in Department 6, Position 2.
FUNC **getProductionStaffRoster**	`getProductionStaffRoster(conn) → list`	Returns all workshop staff with their active order count, allowing managers to balance workload.
CLASS **ProductionAttachment**	`createAttachment(orderID, stage, note, url, user, conn)`	Logs image proofs and completion notes for each completed assembly stage.

production/models/productionoutput.py

QUALITY ASSURANCE (QA) INSPECTION & DEFECT HANDLING

Class & Methods	Signature	Description & Engineering Purpose
CLASS **ProductionOutput**	`recordOutput(...)`	Records official inspection results: `Passed` (credits finished goods, marks order 100% Completed), `Rework` (returns to 75%), or `Failed` (scrapped units).
METHOD **getOrderInspectionSummary**	`(orderID, conn) → dict`	Aggregates cumulative passed vs scrapped units for an order to calculate yield rates.

production/models/productioncost.py

JOB ORDER COSTING (ACTUAL MATERIALS + DIRECT LABOR)

Class & Methods	Signature	Description & Engineering Purpose
CLASS **ProductionCost**	`ProductionCost(data_dict)`	Cost sheet entity wrapping `prod_productioncost`. Tracks material cost, direct labor, and finished unit costs.
METHOD **calculateActualMaterialCost**	`(orderID, conn) → Decimal`	Sums the exact monetary value of raw materials issued from inventory: `∑(quantity_issued × unit_cost)`.
METHOD **calculateCostPerGoodUnit**	`(orderID, totalCost, conn) → Decimal`	Divides total batch cost by passed QA units: `total_cost / passed_units` (Finished Unit Cost).
METHOD **saveCost**	`(orderID, materialCost, laborCost, overheadCost=0.00, conn)`	Inserts or updates the batch cost sheet. Enforces `overhead_cost = 0.00`, ensuring overhead is handled by Sales.

5. HTTP Controller & Route Endpoints (routes.py)

production/routes.py — HTTP REQUEST ENDPOINTS & PRESENTATION LOGIC

Endpoint & Methods	URL Pattern	Description & Controller Actions
ROUTE **login / logout**	/login, /logout	Handles session login, invokes check_ip_security and record_failed_login, clears session cookies upon sign out.
ROUTE **updateAccountProfile**	/account/update-profile	POST endpoint updating name and phone number for the authenticated operator.
ROUTE **changeAccountPassword**	/account/change-password	POST endpoint validating current password and updating the bcrypt hash with a 30-day cooldown.
ROUTE **index**	/ (GET)	Master Executive Dashboard: renders active work order counts, stockout warnings, and 4-week production output bar charts.
ROUTE **furnitureList / addFurniture / edit / delete**	/furniture, /furniture/*	Product Master catalog controller: manages furniture designs, categories, and target prices.
ROUTE **materialList**	/materials (GET)	Raw materials inventory viewer: displays stock on hand, units, categories, and low-stock alerts.
ROUTE **bomList / addBomItem / removeBomItem**	/bom, /bom/add-item	Engineering BOM editor: dynamically calculates unit material costs and updates catalog baseline costs.
ROUTE **orderList / orderDetail / createOrder / delete**	/orders, /orders/<id>	Work Order controller: tracks order registry, planned batch quantities, BOM readiness, and deletion safeguards.
ROUTE **planningList / updateOrderSchedule**	/planning	Production Planning board: updates start dates, due dates, and binds orders to workshop craftsmen.
ROUTE **requirementList / releaseOrder**	/requirement	Material readiness checker: validates component stock and advances orders from Draft to Released.
ROUTE **issuanceList / issueOrderMaterials**	/issuance	Warehouse dispatch: deducts stock from inventory, advances order to In Progress, and auto-starts monitoring.
ROUTE **monitoringList / updateMonitoringProgress**	/monitoring	Workshop floor tracker: workers update assembly stages (0% → 90%) and upload completion photo proofs.
ROUTE **finishedGoodsList / recordInspection**	/finished-goods	Quality Inspection station: records Passed units (100% Complete), Rework (75% rollback), or Scrapped.
ROUTE **costingList / saveOrderCost**	/costing	Production Costing sheet: logs actual direct labor, calculates finished unit cost, and saves cost sheets.
ROUTE **reportView**	/reports (GET)	Executive Reports: batch cost summary, raw material consumption logs, and printable executive sign-off sheets.

> **Developer Summary:** Every database mutation in this application runs inside an atomic transaction managed by db.openTransaction(). If any server error or database disconnection occurs during execution, PostgreSQL safely rolls back all

operations, guaranteeing zero corrupted or orphaned records.